

# Lifecycle Hardening of Immutable Edge Appliances under Power and Connectivity Instability

Dr. S. Suresh Kumar / ScholarMaster Research Group  
Swarnandhra College of Engineering & Technology (Autonomous)

**Abstract**—Edge-based academic installations are typically unmonitored embedded appliances that are placed in an environment where infrastructure resources are limited. The ability to operate in these conditions is mainly related to the degree of reliability of the operating system and updates rather than the accuracy of the applications. Mostly because the devices need to function reliably under varying loads, which is a limiting factor in their adoption for institutional academic settings. Therefore, in such cases, the OS configuration and updates are becoming more critical to ensuring the overall performance and functionality rather than the underlying application. This study presents an invariable edge deployment architecture targeting the enhanced resilience to corruption and self-healing capabilities in the presence of power and network supply fluctuations. We propose an Appliance Invariance Model that utilizes a read-only OverlayFS rootfs, thus isolating all writes in the system to either volatile memory or persistent storage in the form of versioned containers. An extended safety is achieved through watchdog-based hardware monitoring of inference queue liveness, as well as a Blue/Green OTA update procedure with an automated rollback mechanism. A controlled testbed subject to frequent hard power cycles and update failures shows reliable reboots and automated recovery. The resulting system shows the potential improvements in the reliability of distributed academic embedded appliances to be gained by disciplined operating-system immutability and automated lifecycle control.

**Keywords**—Immutable Appliance Architecture, Lifecycle Hardening, OverlayFS Invariance, Bounded Recovery, OTA Rollback, Zero-Touch Provisioning.

\begin{abstract}

Edge-based academic installations are typically unmonitored embedded appliances that are placed in an environment where infrastructure resources are limited. The ability to operate in these conditions is mainly related to the degree of reliability of the operating system and updates rather than the accuracy of the applications. Mostly because the devices need to function reliably under varying loads, which is a limiting factor in their adoption for institutional academic settings. Therefore, in such cases, the OS configuration and updates are becoming more critical to ensuring the overall performance and functionality rather than the underlying application.

This study presents an invariable edge deployment architecture targeting the enhanced resilience to corruption and self-healing capabilities in the presence of power and network supply fluctuations. We propose an Appliance Invariance Model that utilizes a read-only

OverlayFS rootfs, thus isolating all writes in the system to either volatile memory or persistent storage in the form of versioned containers. An extended safety is achieved through watchdog-based hardware monitoring of inference queue liveness, as well as a Blue/Green OTA update procedure with an automated rollback mechanism.

A controlled testbed subject to frequent hard power cycles and update failures shows reliable reboots and automated recovery. The resulting system shows the potential improvements in the reliability of distributed academic embedded appliances to be gained by disciplined operating-system immutability and automated lifecycle control.

\end{abstract}

\begin{IEEEkeywords}

Immutable Appliance Architecture, Lifecycle Hardening, OverlayFS Invariance, Bounded Recovery, OTA Rollback, Zero-Touch Provisioning.

\end{IEEEkeywords}

## INTRODUCTION

The author submits that the primary thesis is that the development of non-modifiable appliances significantly contributes to optimal functioning under unstable supply voltages. Upon the initial deployment in abandoned environments, such devices' functioning is predominantly determined by the underlying operating system's update security and reliability rather than the inherent application stability [Ref].

### Operational Fragility of Mutable Edge Hosts

A conventional Linux system with a writable root filesystem will develop configuration drift due to mutable updates at the level of individual packages. This is caused by arbitrary writes and subsequent state mutations. The result of such instability is that the system becomes more susceptible to filesystem inconsistencies on every reboot following a sudden power loss and every update which introduces new dependencies [Ref]. Unattended devices cannot rely on manual remediation—intermittent infrastructure conditions (unstable power, wireless congestion, restricted physical access) destabilize traditional deployment assumptions. Lifecycle instability emerges structurally from mutability, not merely from poor maintenance practices.

### Contributions and Subsystem Scope

This paper details an immutable, containerized OS architecture designed for Zero-Touch administration. By enforcing strict separation between a read-only host OS and containerized inference logic, we ensure that the edge node can survive power loss, recover from software hangs, and securely update its models without human intervention. Specifically:

- We formalize an Appliance Invariance Model that confines all runtime mutations to volatile memory or versioned container layers.
- We implement Blue/Green OTA rollback as a bounded-failure containment mechanism.
- We integrate hardware watchdog recovery for liveness enforcement under scheduler exhaustion.
- We validate autonomous recovery across 50 simulated hard power-cycles.

Within the ScholarMaster architecture, this subsystem enforces immutable lifecycle behavior, hardens OTA recovery, and stabilizes unattended appliance deployment only. It does not optimize flash endurance (Paper 12), validate runtime enforcement semantics (Paper 18), optimize federated adaptation (Paper 13), derive formal verification theorems (Paper 21), or define architectural privacy doctrine (Paper 17).

## FAILURE-STATE TAXONOMY AND RELATED WORK

To analyze deployment reliability systematically, we categorize the primary failure classes encountered in unattended edge appliances:

- **Power-Loss Corruption:** Unclean shutdowns mid-write causing OS unbootability (addressed via Appliance Invariance and OverlayFS).
- **Kernel Deadlock / Scheduler Exhaustion:** Total system freezes where software supervisors fail (addressed via Hardware Watchdog integration).
- **Application Crash:** Container-level segfaults or memory leaks (addressed via daemon process supervision and cgroup isolation).
- **Malicious Update Injection:** Poisoned firmware or rogue container deployment (addressed via TPM Secure Boot and pre-flight OTA validation).
- **Network Partition:** Multi-day loss of cloud connectivity (addressed via priority-aware buffering).

### MLOps for Edge Devices

Although MLOps is well-established for cloud infrastructure, the same practices have been less explored for distributed edge deployments. Specifically, we note that Sculley et al. [Ref] identified "hidden technical debt" in ML systems which proves particularly challenging in the context of distributed and constrained hardware lacking the reliability of data-center grade power supplies. Edge deployment pipelines are typically not battle-tested in the same way as their cloud counterparts and have to tolerate "dirty" power. We adopt concepts from cloud-native container orchestration [Ref] and apply them within the embedded Linux environment using kernel isolation and daemons supervision as a foundation.

### Embedded Reliability and Filesystems

Flash-based storage systems pose new reliability challenges for devices which are subject to power instability (as is the case for many embedded systems) [Ref]. Journaling filesystems (e.g., Ext4) protect against some sources of metadata corruption, but are not immune to failures in the underlying flash translation layer during an active block write [Ref]. The use of structural immutability for the purpose of defining read-only root structures is promising in terms of extending the lifespan of flash storage [Ref], but carries implications for volatile memory management in high-throughput, log-intensive edge appliances, which could cause memory-swap thrashing and OOM (out-of-memory) cascades if not properly handled.

## APPLIANCE INVARIANCE MODEL

The proposed deployment model treats the edge node as an immutable appliance instead of a conventional mutable Linux host.

The system architecture and end-to-end data pipeline are defined by the broader architectural series; this work focuses on the lifecycle invariance properties of the deployment substrate. The deployment model intentionally avoids traditional package-manager operations such as remote apt-based upgrades executed through SSH. Instead, the host OS is treated primarily as a minimal read-only container runtime environment. All software dependencies, drivers, analytic logic, and neural network models are packaged into immutable Docker images [Ref].

### Formal Invariant

The architectural principle underlying this deployment strategy is formalized as the Appliance Invariance Model. Let  $H$  represent the lower-layer persistent filesystem state,  $C$  represent the containerized application state, and  $V$  represent the volatile overlay state. The core system invariant is:

$$\text{Equation: } \forall \text{forall } t,; H_{\{\text{persistent}\}}(t) = H_0$$

This invariant expresses the design principle that the persistent host OS state remains unchanged during routine operation. Invariance is enforced through mutation confinement: runtime mutations are restricted either to volatile memory ( $V$ )—which is destroyed on power loss—or to versioned container layers ( $C$ ) managed through controlled OTA transitions. The difference is crucial: The persistent immutable state ( $H_0$ ) - this is such state which is never, ever written during normal operations. The volatile operational state ( $V$ ), into which all the writes go, serves as the working area, the place where all the computation happens. The state of the containerized application ( $C$ ) can be changed only through authorized OTA upgrades which implies that the state transitions are controlled and verifiable. Finally, the runtime state changes only inside memory (RAM). This way, any corruption is confined within its space and the power cycle resets it to a clean state. This prevents accumulation of filesystem changes on every reboot.

### Immutability Layer Diagram

[FIGURE: Architectural Pipeline / Validation Curves]

}

\caption{Appliance Invariance Architecture. The persistent lower layer ( $H_0$ ) is never written to during normal operation. All runtime modifications are made either in the volatile UpperDir (RAM) or within versioned container layers. Upon power-down, only the volatile layers are lost.}

```
\label{fig:overlay}  
\end{figure}
```

This architectural decoupling ensures that the environment running the inference code is reproducible across fleet nodes provisioned from identical base images, standardizing deployment configurations and eliminating deployment friction.

## RELIABILITY ENGINEERING

System reliability emerges from the interaction of several complementary mechanisms implemented at the operating-system level.

### Process Supervision

The inference engine is operationalized as a system daemon using strict process-orchestration policies [Ref]. The unit configuration (Listing 1) defines strict restart policies. The OOMScoreAdjust directive explicitly protects the primary Docker daemon and SSH server from the Linux Out-Of-Memory (OOM) killer, shifting most memory exhaustion risk to the individual user-space containers which can be safely restarted without bringing down the host OS or losing network connectivity.

[FIGURE: Architectural Pipeline / Validation Curves]

### Hardware Watchdog Timer (WDT) Integration

Software-level process supervision alone cannot recover from all classes of runtime failure. To address this, we enable the hardware Watchdog Timers (WDT) integrated into commodity ARM-based edge compute modules to provide hardware-level liveness enforcement for appliance survivability [Ref]. The watchdog ensures recovery under liveness failure; detailed runtime enforcement semantics and invariant verification are addressed in the architectural verification layer of the broader series.

### Read-Only Filesystem (OverlayFS) Mechanics

Filesystem immutability is implemented using the OverlayFS union filesystem provided by the Linux kernel. To substantially reduce the likelihood of storage corruption during rolling blackouts, the root filesystem is mounted as read-only using OverlayFS.

The physical storage acts as the immutable Lower Directory. A volatile tmpfs (RAM disk) is mounted as the Upper Directory. The operating system interacts with the Merged View. All system writes (logs, temp files, state changes) are seamlessly intercepted by the Linux kernel and written exclusively to the volatile RAM disk.

Under this configuration, abrupt power removal destroys only the volatile UpperDir residing in RAM. Because persistent storage is mounted read-only during normal operation, boot-sector and base OS corruption events were not observed during evaluated power-cycle tests. OverlayFS isolation confines volatile churn to bounded RAM—corruption domains become spatially limited, and reboot semantics inherently restore clean operational state. This architecture improves bounded recoverability rather than guaranteeing absolute corruption impossibility; hardware-level flash failures, bootloader corruption, and power-delivery anomalies remain outside the OverlayFS protection boundary.

### RAM-Disk Exhaustion Mitigation

To reduce the risk of RAM exhaustion, log output is aggressively bounded. System and container logs are capped, and cgroup-based memory ceilings are applied to inference containers. Because operational telemetry is streamed off-device when connectivity permits, deep local log retention is intentionally avoided.

## OTA UPDATES & FLEET MANAGEMENT

Maintaining a distributed fleet of edge appliances requires deterministic and failure-tolerant update mechanisms. Deploying

updated neural network weights, system dependencies, or application logic across a fleet requires fail-safe OTA mechanisms [Ref]. We implement a Blue/Green deployment state machine leveraging container APIs to provide deterministic cutovers with minimal service disruption [Ref]. The Blue/Green mechanism functions as a bounded-failure containment mechanism: OTA rollback constrains update blast radius, pre-flight validation limits irreversible deployment transitions, and deterministic reversion improves unattended survivability.

### Deterministic Cutover Formalization

We define the update timing constraints as follows. Let  $T_{\text{pre}}$  be the validation time,  $T_{\text{cutover}}$  be the container swap execution, and  $T_{\text{rollback}}$  be the revert time.

In practice, the cutover process is designed to avoid user-visible downtime. The currently active container (Blue) continues serving until the incoming container (Green) passes health checks. Once it does, hardware access is redirected back to the new instance. As can be seen, there were only a few seconds of downtime as the switch from one instance occurred, but no interruptions to the service were reported during the test runs.

The state machine operates as follows:

- Pull (Delta): The device downloads the new container image (Green) alongside the running container (Blue). To conserve constrained academic network bandwidth, we utilize registry delta-compression, pulling only modified layers.
- Pre-Flight Test: Green is started without hardware camera bindings. It executes an internal self-test (verifying weight checksums, loading libraries, and allocating runtime contexts).
- Cutover: If the self-test passes, the supervisor tears down Blue, binds the hardware to Green, and elevates it to primary.
- Automated Rollback: If Green crashes or fails to emit a successful inference heartbeat within 60 seconds, it is destroyed, and Blue is instantly restarted.

## OPERATIONAL CONFIGURATION POLICIES

The appliance configuration translates systemic design constraints into OS-level strictures. We enforce compliance via an automated deployment checklist applied to the base image:

- Volatile Mount Enforcement: `/var/log`, container volumes, and local telemetry buffers are all mounted to tmpfs, which ensures that they can't be persisted to physical media. This is enforced by

checking their mounts during the boot process.

- Swap Disablement: swapoff -a is enforced. This guarantees that application memory buffers are never inadvertently paged to physical disk by the OS memory manager under load.
- Encrypted Keystore: Any persistent identity configuration keys or certificates required for network provisioning are stored on encrypted partitions and managed strictly through secured key services.

## NETWORK & DATA RESILIENCE

Edge appliances are often expected to operate in campus networks where they can be subject to long network partitions. During such periods of downtime, telemetry data is gathered in volatile memory on the edge appliance and later sent once connectivity is re-established. The buffering mechanism stores the telemetry data in an SQLite instance that is resident in memory.

## DEFENSE-IN-DEPTH SECURITY ARCHITECTURE

Devices operating on open campus networks require robust physical and transport-layer security configurations to mitigate the risk of unauthorized firmware modification or network impersonation [Ref].

### Zero-Touch Provisioning (ZTP) Sequence

In order to achieve the Zero-Touch administration, devices are onboarded automatically. Additionally, cryptographic lifecycle management is automated as well using Trusted Platform Module (TPM 2.0) [Ref] hardware security module available in edge board.

On first boot, the device contacts the provisioning server. The device uses the factory fused TPM endorsement key (EK) to assert its hardware identity to the provisioning server over PKCS#11 interface. On successful authentication, the server issues a unique short lived X.509 certificate to the device. The private key of this certificate is generated inside the TPM and not accessible. The device retrieves its operational configuration.

### Mutual TLS (mTLS) & Secure Boot

After Zero-touch provisioning (ZTP), all communication with the central fleet manager is secured by Mutual TLS (mTLS) using TLS 1.2 or higher [Ref]. During this step, the edge node is authenticated, and the server is authenticated with the edge node using a TPM-backed certificate.

The TPM-backed certificate lifecycle reduces the risks of extraction of a private key by relying on a secure hardware module to store it. Similarly, Secure Boot decreases the probability that an arbitrary kernel or bootloader will be run on the device. These measures are all about lifecycle integrity: trusting that the deployment to be operated on is authentic reduces the risks of exfiltration, tampering, or unauthorized updates. In more detail, the threat modeling process is covered in later sections of this series, so we have left out some specifics that would make this post even longer than it already is.

## FULL-STACK OBSERVABILITY PIPELINE

Operational reliability across a distributed fleet requires continuous system observability. The fleet-management layer implements a pull-based monitoring architecture built around Prometheus [Ref], optimized for intermittent edge connectivity.

### Metrics Schema and Local Buffering

A custom Python prometheus-client exporter runs on each node, exposing hardware and application metrics. Metrics are categorized into liveness, thermal, and network-stability domains, enabling fleet-wide anomaly detection via threshold-based alerting.

During network partitions, the central Prometheus server cannot scrape the edge nodes. To prevent metric loss, the edge nodes utilize a

local Prometheus Pushgateway or remote-write buffer.

High-cardinality metrics are downsampled locally during outages to conserve memory, ensuring that essential fleet health data is preserved and successfully transmitted once connectivity is restored.

## FLEET LIFECYCLE VALIDATION

The reliability mechanisms described above were evaluated through a controlled lifecycle validation testbed. Validation was conducted on a controlled testbed designed to simulate fleet conditions. We evaluated the critical lifecycle mechanisms (OTA, Watchdog, Provisioning) independently of inference performance. Results reflect this specific hardware configuration and may vary depending on flash controller firmware.

### Power Failure Recovery Validation

To evaluate corruption resistance under the Power-Loss Failure class, we subjected the OS architecture to 50 simulated hard power-cycles, utilizing power cuts randomized between 0.5–3.0s intervals during peak I/O stress tests via automated relay switching.

**TABLE: Power Failure Resilience Test Results (n=50 cycles)**

Within the evaluated deployment conditions, across 50 induced abrupt power interruptions under active I/O stress, no boot-blocking corruption events were observed in the OverlayFS configuration. In contrast, the standard writable Ext4 configuration required manual intervention in 6 cases. While the sample size is limited and recovery timing depends on board and bootloader characteristics, these results are consistent with improved recoverability through mutation confinement. Long-duration fleet behavior under multi-year continuous operation remains deployment-dependent.

### OTA Rollback Validation

To test the resilience against the Malicious/Corrupt Update Failure class, we intentionally pushed a poisoned container image (configured to segfault upon initialization) to the edge node.

- Detection Time: 4.2 seconds (Pre-flight test failure).
- Rollback Execution: 2.8 seconds (Green destroyed, Blue resumed).
- Total App Downtime: No user-visible interruption was observed; internal container handoff latency remained below measurement granularity (approx 100 ms).

This confirms that corrupted model updates cannot permanently disable the appliance in the field.

Hardware Watchdog (WDT) Recovery

The hardware watchdog was able to reboot the system after a fake scheduler exhaustion. The initial test involved too aggressive inactivity watchdog which resulted in two false reboots during the initial phase of the test. This is why the \Delta t had to be re-adjusted. The total reboot time, from the moment the system freezes to the point where the system is ready to accept commands was less than 1 minute in the given test environment. Note that this time can vary depending on the hardware and a particular bootloader implementation.

Zero-Touch Provisioning Speed

From the moment a factory-reset device is connected to power and the network, the automated certificate exchange, image pulling, and initialization sequence required an average of 3 minutes and 15 seconds to reach an active monitoring state.

DEPLOYMENT ECONOMICS (BOM)

To evaluate physical scale-out feasibility, we detail the per-node hardware Bill of Materials (BOM) in Table \ref{tab:bom}. Cost modeling beyond unit CapEx is outside the scope of this paper.

TABLE: Per-Node Hardware Bill of Materials (CapEx Snapshot)

By enforcing the OverlayFS read-only root and volatile RAM-disk logging, physical wear on the storage medium is constrained. Quantitative flash endurance analysis is presented separately in the storage reliability layer of the broader series.

Limitations

- The reliability mechanisms expose several structural limitations:
- Finite RAM-Disk Capacity: The tmpfs upper layer is bounded by physical memory; unbounded log emission during sustained network partitions triggers OOM failures if not strictly rotated.
  - Firmware Protection Limits: OverlayFS immutability protects the OS root but does not prevent malicious lower-level bootloader or baseband firmware modifications.
  - TPM Hardware Dependence: The Zero-Touch Provisioning sequence relies on hardware TPM 2.0, limiting portability to boards lacking this module.
  - Flash Controller Variability: While root immutability reduces write amplification, proprietary SD card wear-leveling algorithms introduce hardware-specific variance outside OS control. Quantitative flash endurance modeling is addressed separately in Paper 12.
  - Limited Fleet Scale: Validation was conducted on a controlled testbed; fleet-scale behavior across heterogeneous hardware and network conditions remains unvalidated.
  - Controlled Power-Failure Simulation: Power cycles were simulated via automated relay switching; real-world brownouts, voltage sag, and transient instability patterns may differ.
  - Absence of Multi-Year Observation: Long-duration appliance behavior under continuous operation has not been evaluated; filesystem aging, SD-card degradation, and bootloader drift may emerge over extended horizons.
  - Watchdog False Positives: During threshold tuning, overly aggressive inactivity detection caused unintended reboots, indicating sensitivity to calibration under variable workloads.

CONCLUSION

This work analyzed the lifecycle engineering required to deploy resilient edge appliances in infrastructure-constrained environments. The contribution focuses on immutable-state recovery, bounded deployment failure containment, and autonomous appliance restoration.

By combining a read-only OverlayFS root, containerized Blue/Green OTA rollback, hardware watchdog supervision, and TPM-backed provisioning, the architecture demonstrated stable behavior across repeated power interruptions, simulated update corruption, and induced scheduler exhaustion within the evaluated test environment. Mutation confinement through OverlayFS ensures that corruption domains are spatially bounded and reboot semantics restore clean operational state.

Within the ScholarMaster architecture, this framework operates exclusively as the immutable deployment lifecycle and appliance-hardening subsystem—enforcing recovery boundaries and stabilizing unattended deployment without optimizing flash endurance (Paper 12), validating runtime enforcement semantics (Paper 18), managing federated adaptation (Paper 13), or deriving formal verification theorems (Paper 21).

\subsection\*{Reproducibility}

A reference prototype was implemented for evaluation purposes. Implementation details are available upon request.

REFERENCES

[1] C. Mohan et al., "ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," ACM TODS, vol. 17, no. 1, pp. 94-162, 1992.

[2] P. England et al., "A Trusted Open Platform," IEEE Computer, vol. 36, no. 7, pp. 55-62, 2003.

[3] A. Avizienis et al., "Basic Concepts and Taxonomy of Dependable and Secure Computing," IEEE Transactions on Dependable and Secure Computing, vol. 1, no. 1, pp. 11-33, 2004.

[4] A. Mathur et al., "The new ext4 filesystem: current status and future plans," in Proc. Ottawa Linux Symposium, 2007, pp. 21-33.

[5] T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, 2008.

[6] J. Koopman, Better Embedded System Software, Drumnadrochit Education, 2010.

[7] H. Zheng et al., "Understanding the Robustness of SSDs under Power Fault," in Proc. USENIX FAST, 2013, pp. 271-284.

[8] OASIS, "MQTT Version 3.1.1 Specification," OASIS Standard, 2014.

[9] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," Linux Journal, 2014.

[10] C. Boettiger, "An introduction to Docker for reproducible research," ACM SIGOPS Operating Systems Review, vol. 49, no. 1, pp. 71-79, 2015.

[11] J. Lee et al., "F2FS: A New File System for Flash Storage," in Proc. USENIX FAST, 2015, pp. 273-286.

- [12] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," in Proc. NeurIPS, 2015, pp. 2503-2511.
- [13] S. Kuppusamy et al., "Diplomat: Using Delegations to Protect Community Repositories," in Proc. USENIX NSDI, 2016, pp. 567-581.
- [14] W. Shi et al., "Edge Computing: Vision and Challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, 2016.
- [15] B. Burns et al., "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50-57, 2016.
- [16] M. Satyanarayanan, "The Emergence of Edge Computing," Computer, vol. 50, no. 1, pp. 30-39, 2017.
- [17] G. Ananthanarayanan et al., "Real-Time Video Analytics: The Killer App for Edge Computing," IEEE Computer, vol. 50, no. 10, pp. 58-67, 2017.
- [18] D. Crankshaw et al., "Clipper: A Low-Latency Online Prediction Serving System," in Proc. USENIX NSDI, 2017, pp. 613-627.
- [19] J. Turnbull, Monitoring with Prometheus, Turnbull Press, 2018.
- [20] M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," IEEE Data Eng. Bull., vol. 41, no. 4, pp. 39-45, 2018.
- [21] Z. Zhou et al., "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," Proc. of the IEEE, vol. 107, no. 8, pp. 1738-1762, 2019.